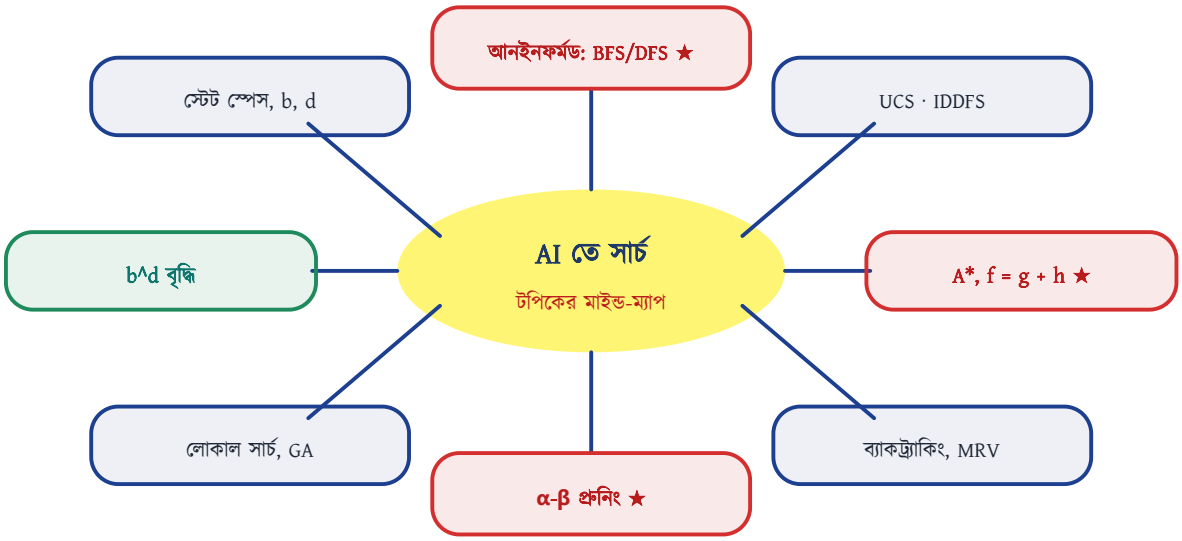
AI এ সার্চ টেকনিক

স্টেট স্পেস · BFS/DFS · UCS/IDDFS · গ্রিডি ও A* (f = g + h) · মিনিম্যাক্স ও α-β · ব্যাকট্র্যাকিং: ১২ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = মূল ধারণা

সিলেবাসের যে অংশ

কৃত্রিম বুদ্ধিমত্তার সিলেবাসে লেখা: "পুরোনো ও নতুন AI কৌশলের পরিচিতি; জ্ঞান উপস্থাপন; প্রপোজিশনাল ও ফার্স্ট অর্ডার লজিক; **AI তে সার্চ টেকনিক**; প্রবণতাভিত্তিক যুক্তি; প্রাকৃতিক ভাষা প্রক্রিয়াকরণ।" এই নোটবুক ঠিক সেই লাইনটিই আলোচনা করে। জ্ঞান উপস্থাপন ও লজিকের নিজস্ব নোট আছে; প্রবণতাভিত্তিক যুক্তি ও NLP আসে এই লেসনের পরে।

হাই-ইল্ড তালিকা

① নোড = অবস্থা, এজ = কর্ম
② BFS = কিউ, DFS = স্ট্যাক
③ UCS = g-এর প্রায়োরিটি কিউ
④ A*: f(n) = g(n) + h(n)
⑤ অ্যাডমিসিবল: h(n) ≤ h*(n)
⑥ α-β সেরা ক্ষেত্রে O(b^(d/2))
⑦ CSP: MRV + ফরওয়ার্ড চেকিং
⑧ সম্পূর্ণ ≠ অপটিমাল

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

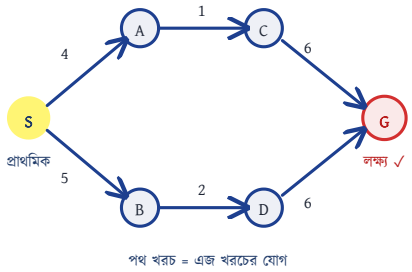
সূচিপত্র

✓ স্টেট স্পেস হিসেবে সার্চ ... পৃ. ২
✓ অগ্রসর ও পশ্চাৎ অনুসন্ধান ... পৃ. ৩
✓ UCS, ডেপথ-লিমিটেড, IDDFS ... পৃ. ৪
✓ অনুমানভিত্তিক: গ্রিডি ও A* ... পৃ. ৫
✓ অ্যাডমিসিবল, সামঞ্জস্য, হিউরিস্টিক ... পৃ. ৬
✓ লোকাল সার্চ, অ্যানিলিং, GA ... পৃ. ৭
✓ মিনিম্যাক্স ও α-β প্রতিরক্ষা ... পৃ. ৮
✓ শর্ত সন্তুষ্টি (CSP) ... পৃ. ৯
✓ সার্চ বাছাই ও খরচ ... পৃ. ১০
✓ রাতের পাতা পৃ. ১১ · অনুশীলন পৃ. ১২

১ স্টেট স্পেস হিসেবে সার্চ

সংজ্ঞা: **স্টেট স্পেস সার্চ** সমস্যাটিকে এমন গ্রাফ হিসেবে মডেল করে যেখানে প্রতিটি **নোড এক মুহূর্তের সম্পূর্ণ অবস্থা** এবং প্রতিটি **এজ একটি কর্ম (অপারেটর)**, যা এজেন্টকে পরবর্তী অবস্থায় নিয়ে যায়। এজেন্ট **প্রাথমিক অবস্থা** থেকে শুরু করে এবং কোনো অবস্থা **লক্ষ্য পরীক্ষায়** উত্তীর্ণ হলেই থেমে যায়।

অবস্থা = একটি ছবি; নোড = ছবি + কীভাবে সেখানে এলাম



প্রতিটি তির = একটি বৈধ কর্ম

ফর্মুলেশনের পাঁচটি অংশ: I প্রাথমিক অবস্থা · A কর্ম · T ট্রানজিশন ফাংশন results(s, a) = s′ · G লক্ষ্য পরীক্ষা · C পথ খরচ $c(s_0, \ldots, s_k)$

পরিভাষা	এক লাইনে অর্থ
স্টেট স্পেস	শুরু থেকে পৌঁছানো সব অবস্থা
ফ্রন্টিয়ার	তৈরি, এখনো বিস্তার হয়নি এমন নোড
ক্লোজড লিস্ট	বিস্তার হওয়া নোড — পুনরাবৃত্তি ঠেকায়
d, b	d = লক্ষ্যের গভীরতা, b = গড় শাখা
g(n)	শুরু থেকে n পর্যন্ত সেরা পথের খরচ

মনে রাখার কৌশল: "**ফ্রন্টিয়ার** হলো অপেক্ষার ঘর, **ক্লোজড লিস্ট** হলো স্মৃতি।" নোড তৈরি হয়ে ফ্রন্টিয়ারে জন্মায়, বিস্তার হয়ে ক্লোজড লিস্টে যায়, আর তার সন্তানরা ফ্রন্টিয়ারে এলে সে বিদায় নেয়।

পরীক্ষার ফাঁদ: (i) **অবস্থা** সম্পূর্ণ বর্ণনা, কোনো ইতিহাস নয়। (ii) ভিন্ন পথে দুটি নোডে একই **অবস্থা** থাকতে পারে; কম খরচেরটিই থাকবে। (iii) লক্ষ্য **পরীক্ষা** =/ লক্ষ্য **অবস্থা**। (iv) b হলো **গড়** শাখা গুণাঙ্ক, সর্বাধিক নয়।

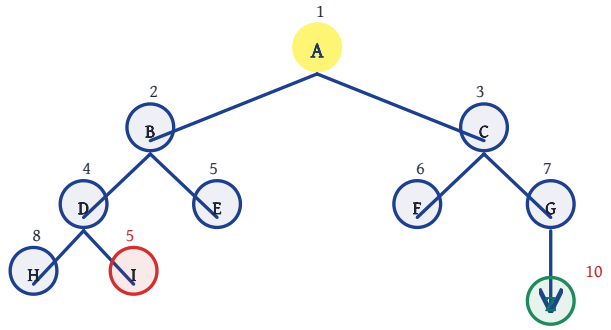
সমাধানকৃত উদাহরণ ✍️ ৮-পাজলের ফর্মুলেশন: **অবস্থা:** ৩ × ৩ বোর্ডে টাইল ১–৮ ও ফাঁকা ঘরের সাজানো। **কর্ম:** পাশের একটি টাইল ফাঁকায় সরানো — ২, ৩ বা ৪টি বৈধ সরণ, তাই b ≤ 8 এবং কার্যকর শাখা গুণাঙ্ক প্রায় **1.67**। **লক্ষ্য পরীক্ষা:** 1 2 3 / 4 5 6 / 7 8 _। **পথ খরচ:** সরণের সংখ্যা। ৯! সাজানোর মধ্যে কেবল **9!/2 = 181 440**টি পৌঁছানো যায়, আর ইনভার্সন পরীক্ষার খরচ $O(n^2)$।

ট্রি সার্চ বনাম গ্রাফ সার্চ: **ট্রি সার্চ** কোনো তালিকা রাখে না, তাই একই অবস্থা বারবার ঘুরতে পারে — তখন সম্পূর্ণতার জন্য b ও d সসীম হতে হবে। **গ্রাফ সার্চ** ক্লোজড লিস্ট রাখে, তাই প্রতিটি অবস্থা একবারই বিস্তৃত হয়, তবে মেমোরি খরচ হয় এবং পরে সস্তা পথ এলে **পুনর্বিস্তারণ (re-expansion)** লাগতে পারে।

IT সংযোগ: একটি প্রোগ্রামের "অবস্থা" মানে মেমোরি ও ভেরিয়েবলের সম্পূর্ণ ছবি, আর রোবটিক্সে কনফিগারেশন স্পেস (configuration space) ঠিক একই ধারণা। যে কোনো সমস্যাকে "অবস্থা + কর্ম + লক্ষ্য পরীক্ষা" আকারে লেখা যায়, সেটিই সার্চ সমস্যা — আর অ্যালগরিদম সেই অবস্থা-গ্রাফেই খোঁজে।

২ অগ্রসর ও পশ্চাৎ অনুসন্ধান

সংজ্ঞা: **আনইনফর্মড (বিমুখী) সার্চ** প্রাথমিক অবস্থা আর লক্ষ্য পরীক্ষা ছাড়া সমস্যার কোনো জ্ঞান ব্যবহার করে না — এটি পদ্ধতিগত, চতুর নয়। BFS ফ্রন্টিয়ারের **সবচেয়ে অগভীর নোড** বিস্তার করে, তাই ফ্রন্টিয়ার একটি **FIFO কিউ**। DFS **সবচেয়ে গভীর নোড** বিস্তার করে, তাই ফ্রন্টিয়ার একটি **LIFO স্ট্যাক** এবং পুনরাবৃত্তিই এর স্বাভাবিক লেখার ধরন।



নীল = BFS ক্রম (FIFO কিউ), লাল = DFS ক্রম (LIFO স্ট্যাক); অগ্রসর স্তরে স্তরে হাঁটে, পশ্চাৎ I-তে মৃত পথে পৌঁছে ফিরে আসে

বৈশিষ্ট্য	অগ্রসর (BFS)	পশ্চাৎ (DFS)
ফ্রন্টিয়ার	FIFO **কিউ**	LIFO **স্ট্যাক**
বিস্তার করে	অগভীরতম নোড	গভীরতম নোড
সম্পূর্ণ	হ্যাঁ, b, d সসীম হলে	**না**
অপটিমাল	হ্যাঁ, **সমান খরচে**	না
সময় / স্থান	O(b^d) / **O(b^d)**	O(b^m) / **O(bm)**
কখন	কম সংখ্যক পদক্ষেপ	মেমোরি স্বল্প

মনে রাখার কৌশল: "**কিউ** চওড়া ও অন্ধ — সব মনে রাখে, কিছুই জানে না। **স্ট্যাক** সরু ও সাহসী — প্রায় কিছুই মনে রাখে না, তবু গভীরে যায়।"

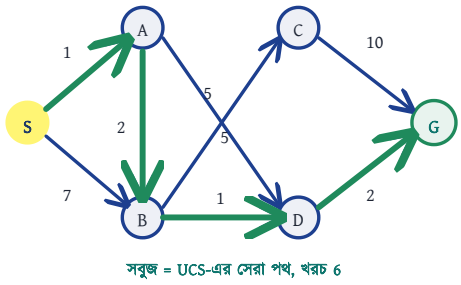
পরীক্ষার ফাঁদ: (i) "BFS সংক্ষিপ্ত পথ খোঁজে" মানে কম **এজ**; ভিন্ন খরচে সবচেয়ে কম এজের পথ সবচেয়ে **ব্যয়বহুল** হতে পারে — তখন UCS। (ii) DFS **ন** সম্পূর্ণ এবং **ন** অপটিমাল। (iii) BFS-এর আসল দাম **স্থান**, সময় নয়। (iv) BFS = কিউ, DFS = স্ট্যাক; **প্রতিটি অনুমানভিত্তিক সার্চ প্রায়োরিটি কিউ** ব্যবহার করে।

সমাধানকৃত উদাহরণ ✍️ কতবার বিস্তার হয়: উপরের ট্রিতে লক্ষ্য Z আছে G-এর নিচে। **BFS** A, B, C, D, E, F, G এবং তারপর Z বিস্তার করে → **8 বার**, মেমোরিতে একটি পূর্ণ স্তর ধরে রাখে। **DFS** A, B, D, H, I (মৃত পথ), E, C, F, G, Z বিস্তার করে → **10 বার**: আগে H ও I-এর মৃত পথে ঢোকে। DFS কেবল বর্তমান পথ রাখে, O(bm), তবে **বেশি** কাজ করে — সম্পূর্ণতার দাম দিতে হয়।

IT সংযোগ: BFS হলো সেই স্তর-ভিত্তিক হাঁটা যা সামাজিক নেটওয়ার্কের "ডিগ্রি অব সেপারেশন", পিয়ার-টু-পিয়ার নেটওয়ার্কের BFS ফ্লাড ফিল ও মেশ নেটওয়ার্কের সবচেয়ে কম হপ-এর রুটিং। DFS হলো DFS স্প্যানিং ট্রি, যা চক্র শনাক্তকরণ, টপোলজিক্যাল সর্টিং, সংযুক্ত উপাদান ও পুনরাবৃত্তি প্রোগ্রামের কল স্ট্যাকের পেছনে।

৩ ইউনিফর্ম-কস্ট, ডেপথ-লিমিটেড ও ইটারেটিভ ডিপেনিং

সংজ্ঞা: **ইউনিফর্ম-কস্ট সার্চ (UCS)** হলো খরচের ওপর BFS: পথ খরচ $g(n)$-এর ওপর সাজানো প্রায়োরিটি কিউ, তাই এটি সবচেয়ে **সস্তা নোড** আগে বিস্তার করে — ডিজকস্ট্রার অ্যালগরিদম। **ডেপথ-লিমিটেড সার্চ (DLS)** হলো গভীরতা l-এ থামানো DFS: $l \geq d$ হলে সম্পূর্ণ, কখনো অপটিমাল নয়। **IDDFS** হলো $l = 0, 1, 2, \ldots$ বাড়িয়ে DLS পুনরাবৃত্তি, তাই এতে **DFS-এর মেমোরি, BFS-এর সম্পূর্ণতা**।



সবচেয়ে সস্তা নোড আগে — প্রতিপক্ষী পথ S-B-C-G-এর খরচ 22

ধাপ	বিস্তার	এরপর ওপেন তালিকা (নোড : g)
1	S (0)	A : 1 · B : 7
2	A (1)	B : **3** (আগে 7) · D : 6
3	B (3)	D : **4** (আগে 6) · C : 8
4	D (4)	G : 6 · C : 8
5	G (6)	লক্ষ্য: S-A-B-D-G

কোনো নোডে সস্তা পথ পাওয়া গেলে সেটি **আবার ঢোকানো** হয় — এই একটি নিয়মই UCS-কে কেবল লোভী না রেখে অপটিমাল করে।

অ্যালগরিদম	সম্পূর্ণ	অপটিমাল	সময়	স্থান
BFS	হ্যাঁ	হ্যাঁ, সমান খরচে	$O(b^d)$	$O(b^d)$
DFS	**না**	**না**	$O(b^m)$	$O(bm)$
UCS (ডিজকস্ট্রা)	হ্যাঁ	হ্যাঁ	$O(b^{(C^*/\varepsilon)})$	$O(b^{(C^*/\varepsilon)})$
DLS	$l \geq d$ হলে হ্যাঁ	না	$O(b^l)$	$O(bl)$
IDDFS	হ্যাঁ	হ্যাঁ, সমান খরচে	$O(b^d)$	$O(bd)$

মনে রাখার কৌশল: "UCS হলো **মিটার টেপসহ BFS** — একই সাফরি, কিন্তু পদক্ষেপ নয় খরচ গোনে। IDDFS হলো **DFS-এর মেমোরি পরা BFS**।" চিহ্ন: b = শাখা গুণাঙ্ক, d = লক্ষ্যের গভীরতা, m = সর্বোচ্চ গভীরতা, C^* = সেরা সমাধানের খরচ, ε = ছোটতম পদক্ষেপ খরচ।

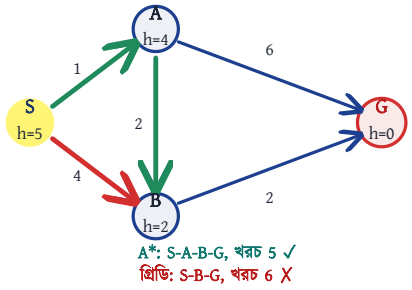
পরীক্ষার ফাঁদ: (i) UCS অপটিমাল শুধু তখনই, যখন প্রতিটি পদক্ষেপের খরচ $\varepsilon > 0$ বা তার বেশি; শূন্য বা ঋণাত্মক এজ প্রমাণটি ভেঙে দেয়। (ii) ওজনযুক্ত গ্রাফে BFS **অপটিমাল নয়** — তাই UCS আছে। (iii) IDDFS কেবল **সমান** খরচে অপটিমাল।

সমাধানকৃত উদাহরণ ✍️ উপরের গ্রাফে UCS: S→A = 1, S→B = 7, A→B = 2, A→D = 5, B→D = 1, B→C = 5, C→G = 10, D→G = 2। UCS S, A, B, D, G বিস্তার করে এবং S-A-B-D-G ফেরত দেয়, খরচ 1 + 2 + 1 + 2 = **6**। প্রতিপক্ষী S-B-C-G-এর খরচ 7 + 5 + 10 = **22** এবং সেটি কখনো খোলাই হয় না। খেয়াল করো: A বিস্তার হওয়ামাত্র B-এর g 7 থেকে 3 হয়ে যায়।

দ্বিমুখী সার্চ: একই সময়ে শুরু থেকে **সামনে** এবং লক্ষ্য থেকে **পেছনে** সার্চ চালিয়ে মাঝখানে মেলানো হয়: b শাখার d-গভীর ট্রি হয়ে যায় দুটি $d/2$-গভীর ট্রি, তাই কাজ পড়ে $O(b^d)$ থেকে প্রায় $O(b^{(d/2)})$।

৪ অনুমানভিত্তিক সার্চ: গ্রিডি বেস্ট-ফার্স্ট ও A*

সংজ্ঞা: **অনুমানভিত্তিক (informed) সার্চ** ডোমেইন জ্ঞান ব্যবহার করে: **হিউরিস্টিক h(n)** n থেকে লক্ষ্য পর্যন্ত খরচের অনুমান। সব বেস্ট-ফার্স্ট অ্যালগরিদম ফ্রন্টিয়ার **প্রায়োরিটি কিউ**-তে রাখে এবং সবচেয়ে ছোট f-এর নোড বের করে; শুধু f কী তা আলাদা: **UCS** f = g(n), **গ্রিডি** f = h(n), **A*** f = **g(n) + h(n)** — g(n) এখন পর্যন্ত খরচ, h(n) বাকির খরচ।



একই গ্রাফ, একই হিউরিস্টিক — শুধু f ঠিক করে কোন পথ ফেরত হবে

অ্যালগরিদম	f(n)	সাজায়	সম্পূর্ণ	অপটিমাল
UCS	g(n)	খরচ পর্যন্ত	হ্যাঁ	হ্যাঁ
গ্রিডি	h(n)	লক্ষ্যের কাছে	না	না
A*	**g(n) + h(n)**	সবচেয়ে সস্তা পথ	হ্যাঁ	হ্যাঁ, h অ্যাডমিসিবল হলে

A*-এর ধর্ম. সময় O(b^d), স্থান O(b^d) — সবচেয়ে খারাপ ক্ষেত্রে BFS-এর চেয়ে ভালো নয়; লাভ আসে **ধ্রুব গুণাঙ্ক** থেকে, কারণ অ্যাডমিসিবল h ট্রির বেশিরভাগ কেটে দেয়। h = 0 বসালে A* নিজেই UCS হয়ে যায়।

সমাধানকৃত উদাহরণ ✍️ একই গ্রাফ, দুই উত্তর: এজ S→A = 1, S→B = 4, A→B = 2, A→G = 6, B→G = 2; h(S) = 5, h(A) = 4, h(B) = 2, h(G) = 0।

গ্রিডি বেস্ট-ফার্স্ট (f = h): S বিস্তার → A (4), B (2); **B** বিস্তার; B থেকে G (0); **G** বিস্তার → S-B-G ফেরত, খরচ 4 + 2 = **6**।

A* (f = g + h): S (f = 5) বিস্তার → A (g 1, f 5), B (g 4, f 6); **A** বিস্তার → B আবার g = 3 < 4 পাওয়া যায়, তাই B হয় g 3, f 5; **B** বিস্তার → G আবার g = 5 < 7 পাওয়া যায়; **G** বিস্তার → S-A-B-G ফেরত, খরচ 1 + 2 + 2 = **5**।

মনে রাখার কৌশল: "**g হলো ইতিমধ্যে দেওয়া বিল, h ডাক্তারের অনুমান, f হলো মোট বিল।**" গ্রিডি কেবল অনুমান দেখে বিল ফেলে দেয়; A* দুটো যোগ করে — তাই গ্রিডি ব্যয়বহুল পথ নিতে পারে।

পরীক্ষার ফাঁদ: (i) শুধু f = h(n) হলো **গ্রিডি**, শুধু f = g(n) হলো **UCS**, f = g·h অর্থহীন। (ii) A*-এর অপটিমালতা h-এর প্রতিশ্রুতি, A*-এর নয়: **বেশি** অনুমান করা h নীরবে অসেরা পথ ফেরত দেয়। (iii) গ্রিডি বেস্ট-ফার্স্ট **অসম্পূর্ণ**। (iv) টাই ব্রেক শুধু কাজ বদলায়, সেরা মান নয়।

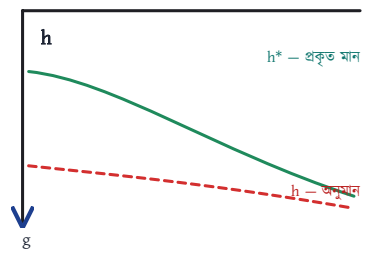
A* আসলে কোথায় চলে: রুট খোঁজা (Google Maps, OSRM), রোবট পাথ পরিকল্পনা, অ্যাসেম্বলি লাইন সাজানো ও টাইপরাইটারের শব্দ পূর্বানুমান। সামঞ্জস্যপূর্ণ (consistent) h-এর সঙ্গে A* আসলে সর্বনিম্ন পথের গতিশীল-প্রোগ্রামিং দৃষ্টিভঙ্গি: প্রতিটি নোড f-এর বর্ণিত ক্রমে একবারই নিশ্চিত হয়।

৫ অ্যাডমিসিবল, সামঞ্জস্যপূর্ণ ও ভালো হিউরিস্টিক

সংজ্ঞা: n থেকে লক্ষ্যে যাওয়ার সবচেয়ে সস্তা পথের প্রকৃত খরচকে **h*(n)** লিখি। কোনো হিউরিস্টিক **অ্যাডমিসিবল (admissible)** যদি সে **কখনো প্রতিরকমিতা করে না**: প্রতিটি n-এর জন্য h(n) ≤ h*(n)। আর তা **সামঞ্জস্যপূর্ণ (consistent)** যদি প্রতিটি এজে **h(n) ≤ c(n, n′) + h(n′)** হয় — অনুমান কর্মের দামের বেশি পড়তে পারে না। সামঞ্জস্যপূর্ণতা অ্যাডমিসিবলতা বোঝায়।

সমস্যা	প্রচলিত হিউরিস্টিক h(n)	অ্যাডমিসিবল?	মন্তব্য
রুট খোঁজা	সরলরেখার দূরত্ব	হ্যাঁ	রাস্তা ঘোরে, তাই দুর্বল
৮-পাজল	ভ্রুক্ষেপে না থাকা টাইল	হ্যাঁ	খুব দুর্বল — ফাঁকা ঘর দেখে না
৮-, ১৫-পাজল	**ম্যানহাটন দূরত্ব**	হ্যাঁ	সারি + কলাম; আদর্শ পছন্দ
স্লাইডিং পাজল	প্যাটার্ন ডেটাবেস	হ্যাঁ	কয়েকটি টাইলের প্রকৃত খরচ
৮-রানী	আক্রমণকারী জোড়া	না	২ দিয়ে ভাগ করলে ঠিক হয়
ট্রাভেলিং সেলসম্যান	বাইরে সবচেয়ে সস্তা সরল সংযোগ	হ্যাঁ	ধারণাগত নিম্নতম সীমা

সমাধানকৃত উদাহরণ ১ ✍️ ৮-পাজল, ম্যানহাটন: শুরু 2 8 3 / 1 6 4 / 7 5 _ , লক্ষ্য 1 2 3 / 4 5 6 / 7 8 _ । ভ্রুক্ষেপে নেই এমন টাইল ১, ২, ৪, ৫, ৬, ৮ → **h = 6**। ম্যানহাটন: টাইল 1 → 1, 2 → 1, 4 → 2, 5 → 1, 6 → 1, 8 → 2 → **h = 8**। প্রতিটি সরণ একটি টাইল এক ঘর নামে, তাই অন্তত এত সরণ বাকি: h* ≥ 8।



অ্যাডমিসিবল অনুমান কখনো প্রকৃত মানের ওপরে ওঠে না

সমাধানকৃত উদাহরণ ২ ✍️ অ্যাডমিসিবল পরীক্ষা: S→A = 2, S→B = 1, A→C = 2, B→C = 4, B→D = 5, C→G = 3, D→G = 2, h(S) = 5, h(A) = 4, h(B) = 4, h(C) = 3, h(D) = 2, h(G) = 0। লক্ষ্য থেকে পেছনে: h*(G) = 0, h*(C) = 3, h*(D) = 2, h*(A) = 5, h*(B) = 7, h*(S) = 7। তুলনা: 5 ≤ 7, 4 ≤ 5, 4 ≤ 7, 3 ≤ 3, 2 ≤ 2, 0 ≤ 0 — **অ্যাডমিসিবল ও সামঞ্জস্যপূর্ণ**।

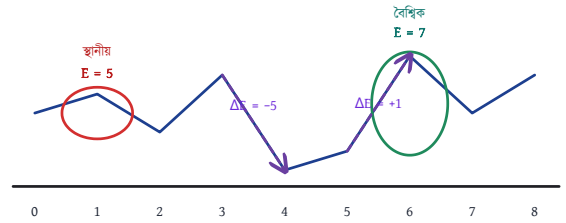
মনে রাখার কৌশল: "**অ্যাডমিসিবল মান কম লক্ষ্য করে।** সামঞ্জস্যপূর্ণতা সাবধানী: কর্মের যা খরচ, তার বেশি অনুমান কমতে পারে না।" স্তর: **সামঞ্জস্যপূর্ণতা ⇒ অ্যাডমিসিবলতা**, উল্টোটা কখনো নয়।

পরীক্ষার ফাঁদ: (i) অ্যাডমিসিবল মানে **কখনো বেশি অনুমান না করা** — h = 0 তাত্ত্বিকভাবে অ্যাডমিসিবল কিন্তু অকার্যকর। (ii) h হলো **বাকির** অনুমান, h* প্রকৃত মান। (iii) **ট্রি সার্চ** A*-এর জন্য অ্যাডমিসিবল h যথেষ্ট, **গ্রাফ সার্চ** A*-এর জন্য সামঞ্জস্যপূর্ণ h দরকার। (iv) ম্যানহাটন দূরত্ব **ভ্রুক্ষেপে না থাকা টাইলের সংখ্যা নয়**।

"ভালো" মানে কী? ✍️: যদি h = c · h* হয়, c ধ্রুব > 0, তবে ভুল δ = h* − h-এর সঙ্গে বাড়ে, আর A* মোটামুটি **শাখা গুণাঙ্ক b^δ-এর একটি আনইনফর্মড সার্চের মতো** আচরণ করে: ৮-পাজলে টাইল-গণনার হিউরিস্টিক আচরণে b ≈ 3, আর ম্যানহাটন b ≈ 1.4।

৬ লোকাল সার্চ: হিল ক্লাইম্বিং, অ্যানিলিং, GA

সংজ্ঞা: **লোকাল সার্চ** কেবল **একটি বর্তমান অবস্থা** রাখে — তাদের গাছ নয় — আর পথ খরচের বদলে **মূল্যায়ন ফাংশন E(s)** দিয়ে তার মান দেখায়। এটি এমন স্থানের জন্য যখন গ্রাফ বানানো যায় না, বিশেষত ধারাবাহিক স্থানের জন্য। **হিল ক্লাইম্বিং** E বেশি এমন প্রতিবেশীতে যায়, আর যেখানে আর ভালো প্রতিবেশী নেই সেখানেই থেমে যায়: একটি **স্থানীয় অনুকূল (local optimum)**, যা বৈশ্বিক নাও হতে পারে।



E(x), x = 0...8; হিল ক্লাইম্বিং x = 1-এ থেমে যায়, অ্যানিলিং খাত টপকায়

x = 3 (E = 6) থেকে প্রতিবেশী x = 2 (3) ও x = 4 (1) দুটোই খারাপ, তাই হিল ক্লাইম্বিং **স্থানীয় সর্বোচ্চ** 6-এ আটকে যায়, যদিও বৈশ্বিক সর্বোচ্চ 7 (x = 6) দুই ধাপ দূরে।

কোথায় ব্যবহার হয়। হিল ক্লাইম্বিং: VLSI লেআউট, চ্যানেল বরাদ্দ। অ্যানিলিং: ৮-রানী, ট্যুর উন্নতি, চিপ প্লেসমেন্ট। জেনেটিক অ্যালগরিদম: অ্যান্টেনা প্লেসমেন্ট।

x	0	1	2	3	4	5	6	7	8
E(x)	4	5	3	6	1	2	7	4	6

পদ্ধতি	কী করে	স্থানীয় অনুকূল?
হিল ক্লাইম্বিং	ভালো প্রতিবেশীতে যায়, না থাকলে থামে	না — আটকে যায়
এলোমেলো রিস্টার্ট	আটকে গেলে নতুন অবস্থা থেকে শুরু, সেরাটি রাখে	হ্যাঁ, ভাগ্যক্রমে
সিমুলেটেড অ্যানিলিং	খারাপ কর্ম গ্রহণের সম্ভাবনা $e^{(-\Delta E/T)}$, T কমে	T বড় থাকলে হ্যাঁ
জেনেটিক অ্যালগরিদম	একটি **জনগোষ্ঠী**: সিলেকশন, ক্রসওভার, মিউটেশন	হ্যাঁ, অনেক প্রার্থী একসঙ্গে বেঁচে থাকে
টাবু সার্চ	গত কয়েকটি কর্ম নিষিদ্ধ করে (স্বল্প মেমোরি)	হ্যাঁ, গর্ত থেকে বের হয়

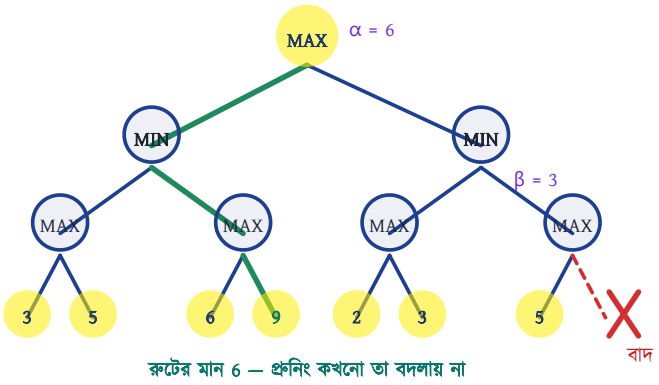
সমাধানকৃত উদাহরণ ✍️ E(x)-এর ওপর অ্যানিলিং: x = 3 (E = 6) থেকে প্রতিবেশী x = 2 (E = 3) ও x = 4 (E = 1) ডাউনহিল, ΔE = -5 ও -3। T = 2 হলে x = 4-এ যাওয়া গ্রহণ হয় সম্ভাবনা $e^{(-5/2)} \approx$ **0.082**-এ — অসম্ভব নয়, তবে কম। x = 4 থেকে x = 5-এ গেলে E বাড়ে 2, তাই তা সরাসরি গ্রহিত হয়; তারপর x = 6 এলে E = 7, বৈশ্বিক সর্বোচ্চ। T → 0 হলে গ্রহণের সম্ভাবনা → 0 এবং অ্যালগরিদম ঠিক হিল ক্লাইম্বিংয়ের মতোই শেষ হয়।

মনে রাখার কৌশল: **"অ্যানিলিং মানে ঠান্ডা খাবার খাওয়া** — খারাপ খাবার থেকে বাঁচতে আগে একটু খারাপই মেনে নিতে হয়। **জেনেটিক অ্যালগরিদম একটি জনগোষ্ঠীর প্রজনন**: ফিটনেস, সিলেকশন, ক্রসওভার, মিউটেশন।"

পরীক্ষার ফাঁদ: (i) হিল ক্লাইম্বিং স্থানীয় সর্বোচ্চে আটকে যাওয়া তার **নিজস্ব সংজ্ঞা**, বাগ নয়। (ii) গ্রহণের সম্ভাবনা T কমলেও কমে, কর্ম খারাপ হলেও কমে। (iii) GA-র কর্মপদ্ধতি **সিলেকশন, ক্রসওভার, মিউটেশন**। (iv) লোকাল সার্চ কোনো **পথ** রাখে না, তাই অপটিমাল প্রমাণ করতে পারে না।

৭ প্রতিপক্ষী সার্চ: মিনিম্যাক্স ও α-β প্রুনিং

সংজ্ঞা: দুই প্রতিপক্ষিও যখন একই অবস্থা ভাগ করে নেয়, তখন গ্রাফটি একটি গেম ট্রি: একদিক চায় উপযোগিতা **বাড়াতে** (MAX), অন্যদিক **কমাতে** (MIN)। **মিনিম্যাক্স** গভীরে প্রথমে ট্রিটি হেঁটে বদলে বদলে প্রতিটি চালে সেরা জবাব দেয়: টার্মিনাল নোডে v(s) = উপযোগিতা, MAX নোডে **সন্তানের সর্বোচ্চ**, MIN নোডে **সর্বনিম্ন**। b শাখা ও d গভীরতায় খরচ **O(b^d)** নোড মূল্যায়ন।



বাম অর্ধেক α = 6 দেওয়ার পর G-তে প্রথম পাতা 5-ই β = 3-এর চেয়ে ভালো, তাই পাতা 8 বাদ পড়ে

নোড	বাম পাতা	ডান পাতা	মান	মন্তব্য
D (MAX)	3	5	5	max(3, 5)
E (MAX)	6	9	9	max(6, 9)
B (MIN)	D = 5	E = 9	**6**	min → রুটে **α = 6**
F (MAX)	2	3	3	max(2, 3) → C-তে **β = 3**
G (MAX)	5	8	—	5-ই β-এর সমান বেশি, তাই পাতা 8 বাদ
C (MIN)	F = 3	G = 5	3	min(3, 5)
রুট (MAX)	B = 6	C = 3	**6**	max(6, 3) = 6

মনে রাখার কৌশল: "**α হলো MAX-এর এ পর্যন্ত সেরা মান, β হলো MIN-এর এ পর্যন্ত সেরা মান।** কোনো নোডের সেরা মান আর প্রতিপক্ষির হাতে যা আছে তা আর হারাতে পারে না — তাই কাটা যায়।" α কেবল MAX নোডে, β কেবল MIN নোডে হালনাগাদ হয়; নোডে α ≤ β সবসময়।

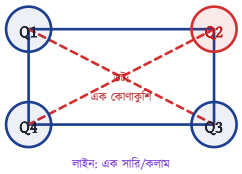
পরীক্ষার ফাঁদ: (i) প্রুনিং **কাজ** বদলায়, **মান** কখনো নয় — সাধারণ মিনিম্যাক্সের সমান মানই ফেরত হয়। (ii) কর্মের ক্রমই আসল: সেরা কর্ম আগে **O(b^(d/2))**, অনিয়ম ব্রতন **O(b^(3d/4))**, সবচেয়ে খারাপ ক্রমে পুরো O(b^d)। (iii) মিনিম্যাক্স ধরে নেয় **উভয় পক্ষই অপটিমাল খেলে**; বাস্তব প্রতিপক্ষি মিনিম্যাক্স খেলে না।

সমাধানকৃত উদাহরণ ✍️ সংখ্যায় সাশ্রয়: পূর্ণ মিনিম্যাক্স ৮টি পাতা মূল্যায়ন করে; উপরের ক্রমে α-β করে 7টি — একটি পাতা সাশ্রয়। গভীরতা বাড়লে প্রভাব বাড়ে: b = 2, d = 10 হলে পূর্ণ মিনিম্যাক্স 2^10 = **1024** পাতা লাগে, আর নিখুঁত ক্রমে α-β প্রায় 2^5 = **32** — একই সময়ে **দ্বিগুণ গভীর** অনুসন্ধান। আধুনিক দাবা ইঞ্জিনে α-β-এর সঙ্গে ট্রান্সপজিশন টেবিল, কিলার মুভ ও মুভ-অর্ডারিং যোগ করা হয়।

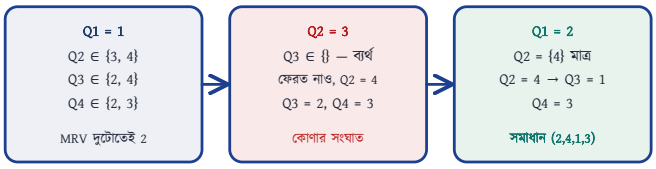
৮ শর্ত সন্তুষ্টি (CSP) ও ব্যাকট্র্যাকিং

সংজ্ঞা: একটি **শর্ত সন্তুষ্টি সমস্যা (CSP)** হলো ত্রৈপক (X, D, C): **চলকের সকল X**, প্রতিটি চলকের **ডোমেইন D**, এবং একসঙ্গে খাটতে হবে এমন **শর্তের সকল C**। **সমাধান** হলো সম্পূর্ণ নিয়োগ (assignment) — কোনো পথ, খরচ বা মধ্যবর্তী পুরস্কার নেই, তাই এখানে BFS বা A* চালানো হয় **না**। আদর্শ সমাধানকারী হলো **ব্যাকট্র্যাকিং সার্চ**: একটি মান বসান, পুনরাবৃত্তি করো, শর্ত ভাঙলে নিয়োগ ফেরত নাও।

উপাদান	অর্থ
চলক, ডোমেইন	যা ঠিক করতে হবে ও তার মান
unary/binary/n-ary	১, ২, k চলকের শর্ত = তার **আর্টি (arity)**
শর্ত গ্রাফ	চলক = নোড, প্রতি জোড়ায় একটি এজ
ব্যাকট্র্যাকিং	বসাও, পুনরাবৃত্তি, **ব্যর্থ হলে ফেরত নাও**
ফরওয়ার্ড চেকিং	প্রতিটি নিয়োগের পরে প্রতিবেশীর ডোমেইন ছেঁকে ফেলো
MRV (fail-first)	পরের চলক = যার **সবচেয়ে কম মান বাকি**
LCV	পরের মান = যা অন্যদের **সবচেয়ে কম** বাঁধে
আর্ক সামঞ্জস্যতা AC-3	প্রতিটি মানের পাশে সমর্থন থাকা দরকার
min-conflicts	লোকাল সার্চ: সবচেয়ে খারাপ সংঘাত ঠিক করো



৪-রানীর গ্রাফ সম্পূর্ণ — প্রতিটি জোড়া শর্তযুক্ত



প্রতিটি নিয়োগের পর ফরওয়ার্ড চেকিং

সমাধানকৃত উদাহরণ ✍️ ৪-রানী: Q1 = 1 দিলে Q2 ∈ {3,4}, Q3 ∈ {2,4}, Q4 ∈ {2,3}: সব ডোমেইনে দুটি মান, তাই MRV যেকোনোটি নিতে পারে। Q2 = 3 করলে Q3 খালি; Q2 = 4 করলে Q3 = 2 বাধ্য, ফলে Q4 = 3 এবং কোণার সংঘাত — অর্থাৎ Q1 = 1 মৃত। **Q1 = 2** দিলে Q2 ∈ {4}, একটিমাত্র মান: MRV জোর করে **Q2 = 4** বসায়, Q3 বাধ্য 1 ও Q4 বাধ্য 3, তাই চার নিয়োগেই সমাধান **(2, 4, 1, 3)**।

মনে রাখার কৌশল: "**MRV সবচেয়ে বাঁধা প্রশ্নটি আগে করে; LCV সবচেয়ে কম বিরোধী উত্তরটি আগে দেখে।** ফরওয়ার্ড চেকিং প্রতিবেশী হারানো সঙ্গে সঙ্গে তার হারানো উত্তর মুছে দেয়।"

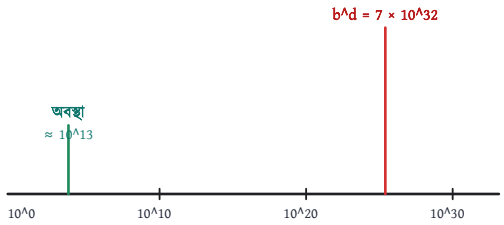
পরীক্ষার ফাঁদ: (i) CSP **পথ অনুসন্ধান নয়**: শুধু চূড়ান্ত নিয়োগই গুরুত্বপূর্ণ। (ii) ফরওয়ার্ড চেকিং **আর্ক সামঞ্জস্যতা নয়** — এটি প্রতিবেশীর ওপরে দেখে না। (iii) MRV চলক বাছে, LCV মান বাছে। (iv) ব্যাকট্র্যাকিং সম্পূর্ণ কিন্তু তাৎপর্যিক; সাধারণ CSP NP-complete।

প্রয়োগ: সুডোকু, টাইমটেবিল, রোস্টার, ক্রিপটারিদমেটিক, শিডিউলিং, কম্পাইলারে রেজিস্টার বরাদ্দ। ৮ × ৮ বোর্ডে সাধারণ ব্যাকট্র্যাকিং প্রথম সমাধানে **10^5** ক্রম নোড লাগে; MRV ও ফরওয়ার্ড চেকিং যোগ করলে তা কয়েকশোতে নেমে আসে।

৯ সার্চ বাছাই ও এর খরচ

অ্যালগরিদম	সম্পূর্ণ	অপটিমাল	সময়	স্থান	ফ্রন্টিয়ার
BFS	হ্যাঁ	হ্যাঁ, সমান খরচে	O(b^d)	O(b^d)	FIFO কিউ
DFS	না	না	O(b^m)	O(bm)	LIFO স্ট্যাক
UCS	হ্যাঁ	হ্যাঁ	O(b^(C*/ε))	O(b^(C*/ε))	g-এর কিউ
IDDFS	হ্যাঁ	হ্যাঁ, সমান খরচে	O(b^d)	O(bd)	প্রতি গভীরতায় স্ট্যাক
গ্রিডি বেস্ট-ফার্স্ট	না	না	O(b^m)	O(b^m)	h-এর কিউ
A*	হ্যাঁ	হ্যাঁ, অ্যাডমিসিবল h	O(b^d)	O(b^d)	g + h-এর কিউ
α-β	হ্যাঁ	হ্যাঁ, একই মান	O(b^(d/2)) সেরা	O(bd)	α, β সীমা

সমস্যা দেখতে যদি এমন হয়...	ব্যবহার করো
কম পদক্ষেপ, সব খরচ সমান	BFS, বা মেমোরি কম হলে IDDFS
গভীর সার্চ, মেমোরি কম	DFS, DLS, IDDFS
রাস্তা, টাকা, জ্বালানি	UCS / ডিজকস্ত্রা
রাস্তা আর মানচিত্র হাতে	A*, সরলরেখার দূরত্ব হিউরিস্টিকে
বোর্ড গেম, প্রতিপক্ষি আছে	মিনিম্যাক্স + α-β
টাইমটেবিল, রোস্টার, সুডোকু	ব্যাকট্র্যাকিং + MRV + ফরওয়ার্ড চেকিং



১৫-পাজল: b = 2.13, d = 100 হলে b^d ≈ 7 × 10^32 নোড, অথচ অবস্থা আছে মাত্র 16!/2 ≈ 1.05 × 10^13

পরীক্ষার ফাঁদ: (i) **সম্পূর্ণ =/ অপটিমাল:** BFS ও IDDFS দুটোই সম্পূর্ণ, কিন্তু সমান খরচে অপটিমাল। (ii) A*-এর O(b^d) একটি **সবচেয়ে খারাপ ক্ষেত্র**; কিছুই A* বক্তিমূলক করে না। (iii) α-β **সময় বাঁচায়, সত্যতা নয়**। (iv) এক সমস্যায় অ্যাডমিসিবল হিউরিস্টিক আরেক সমস্যায় অকার্যকর।

IT সংযোগ: ডিজকস্ত্রা/UCS চালায় নেটওয়ার্ক রাউটিং টেবিল; A* চালায় রাস্তার রাউটিং ও রোবট পাথ পরিকল্পনা; BFS চালায় স্তর-ভিত্তিক ট্রাভার্সাল ও সবচেয়ে কম হপ; IDA* চালায় আদর্শ ১৫-পাজল সমাধানকারী; প্রতিটি সুডোকু সমাধানকারী MRV ও ফরওয়ার্ড চেকিং ব্যবহার করে; α-β প্রতিটি দাবা ও গো ইঞ্জিনে আছে।

সমাধানকৃত উদাহরণ ✍️ হিউরিস্টিক ছাড়া চলে না কেন: ১৫-পাজলে আনইনফর্মড সার্চের সামনের ট্রি প্রায় **7 × 10^32** নোড (b = 2.13, d = 100), অথচ পুরো সমস্যায় মাত্র 16!/2 ≈ **1.05 × 10^13** ভিন্ন অবস্থা আছে। অ্যাডমিসিবল ম্যানহাটন হিউরিস্টিকে কার্যকর শাখা গুণাঙ্ক প্রায় 1.4-এ নেমে আসে — এটাই শেষ হওয়া আর কখনো না শেষ হওয়ার পার্থক্য।

পরীক্ষার আগের রাতের পাতা

তথ্য-তালিকা:

নোড = অবস্থা + পিতৃ + কর্ম + g(n)

b = শাখা, d = লক্ষ্যের গভীরতা, m = সর্বোচ্চ

UCS: g(n)-এর প্রায়োরিটি কিউ

A*: f(n) = g(n) + h(n)

গ্রিডি: শুধু f(n) = h(n)

h = 0 রাখলে A* = UCS

অ্যাডমিসিবল: h(n) ≤ h*(n)

সামঞ্জস্যপূর্ণ: h(n) ≤ c(n,n′) + h(n′)

৮-পাজল: 9!/2 = 181 440, b ≈ 1.67

ম্যানহাটন = সারি + কলাম

α-β: O(b^(d/2)), অনিয়মে O(b^(3d/4))

CSP = (X, D, C); MRV = সবচেয়ে কম মান

অ্যানিলিং: গ্রহণের সম্ভাবনা e^(-ΔE/T)

GA: সিলেকশন, ক্রসওভার, মিউটেশন

MCQ-র জন্য সেরা এক-লাইনের তথ্য

১. BFS-এর ফ্রন্টিয়ার FIFO **কিউ**; DFS = LIFO **স্ট্যাক**

২. BFS কেবল **সমান** খরচে অপটিমাল

৩. DFS ন সম্পূর্ণ, ন অপটিমাল

৪. UCS = ডিজকস্ত্রা, অপটিমাল, ε > 0 লাগে

৫. IDDFS = BFS-এর সম্পূর্ণতা + DFS-এর মেমোরি O(bd)

৬. গ্রিডি বেস্ট-ফার্স্ট: f = h, অপটিমাল নয়

৭. A* অপটিমাল যদি h অ্যাডমিসিবল হয়

৮. সামঞ্জস্যপূর্ণতা ⇒ অ্যাডমিসিবলতা

৯. ৮-পাজলের হিউরিস্টিক ম্যানহাটন দূরত্ব

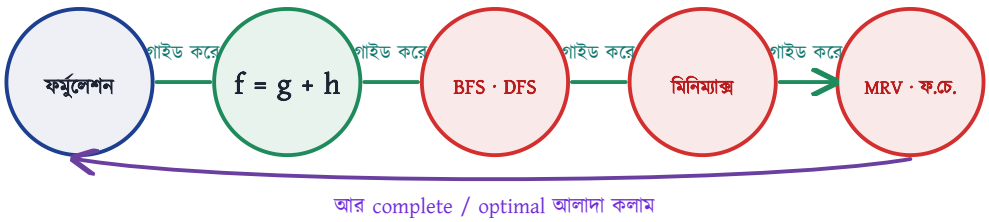
১০. মিনিম্যাক্স: MAX-এ max, MIN-এ min

১১. α-β কাজ কমায়, মান নয়

১২. CSP = চলক, ডোমেইন, শর্ত — কোনো পথ নেই

প্রধান ফাঁদ: BFS অপটিমাল =/ BFS সবসময় অপটিমাল (ওজনযুক্ত গ্রাফে UCS দরকার) · f = h হলো গ্রিডি, f = g হলো UCS, f = g + h হলো A* · h = 0 অ্যাডমিসিবল কিন্তু অকার্যকর · অসীম গভীরতার গ্রাফে DFS-এর গভীরতা সীমা দরকার · হিল ক্লাইম্বিং **স্থানীয়** সর্বোচ্চে থেমে যায় · α MAX নোডে, β MIN নোডে হালনাগাদ হয় · ফরওয়ার্ড চেকিং ডোমেইন ছেঁকে, আর্ক সামঞ্জস্যতা আরও শক্তিশালী।

সব কৌশল একসঙ্গে: "ফ্রন্টিয়ার অপেক্ষার ঘর, ক্লোজড লিস্ট স্মৃতি" · "কিউ চওড়া ও অন্ধ, স্ট্যাক সরু ও সাহসী" · "UCS মিটার টেপসহ BFS; IDDFS DFS-এর মেমোরি পরা BFS" · "g দেওয়া বিল, h অনুমান, f মোট বিল" · "অ্যাডমিসিবল মান কম লক্ষ্য করে; সামঞ্জস্যপূর্ণতা সাবধানী" · "অ্যানিলিং ঠান্ডা খাবার খায়, GA জনগোষ্ঠীর প্রজনন করে"।



একটি ফর্মুলেশন, একটি প্রায়োরিটি নিয়ম, পাঁচটি অ্যালগরিদম — আর দুটি কলাম এক নয়

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. BFS-এর ফ্রন্টিয়ার কী?

ক) LIFO স্ট্যাক　　খ) প্রায়োরিটি কিউ

গ) সার্চ ট্রি　　ঘ) FIFO কিউ

২. A* পরের নোড কী দিয়ে বাছে?

ক) g(n) + h(n)　　খ) g(n) × h(n)

গ) শুধু h(n)　　ঘ) শুধু g(n)

৩. h(n) অ্যাডমিসিবল যখন:

ক) বেশি অনুমান করে খ) কখনো বেশি করে না

গ) সময় ধ্রুব ঘ) প্রতি এজে 1 কমে

৪. নিখুঁত ক্রমে α-β কত নোড দেখে?

ক) O(b × d) খ) O(d^b)

গ) O(b^(d/2)) ঘ) O(b^(2d))

৫. IDDFS কেন পছন্দ করা হয়?

ক) সবসময় অপটিমাল খ) DFS-এর মেমোরি

গ) গভীরতার সীমা লাগে না ঘ) নোড কখনো আরায় না

৬. সাধারণ DFS কখন সম্পূর্ণ?

ক) খরচ সমান হলে খ) গ্রাফ সসীম হলে

গ) h অ্যাডমিসিবল হলে ঘ) গভীরতা সসীম হলে

৭. MIN নোডে মিনিম্যাক্স কী দেয়?

ক) সর্বনিম্ন মান খ) সর্বোচ্চ মান

গ) সমষ্টি ঘ) α + β

৮. CSP-তে MRV কী বাছে?

ক) সবচেয়ে বড় ডোমেইন খ) সবচেয়ে বেশি শর্ত

গ) সবচেয়ে কম মান বাকি ঘ) সবচেয়ে বেশি ডিগ্রি

৯. অ্যানিলিং খারাপ কর্ম গ্রহণ করে:

ক) T কমলে কমে খ) p = e^(-ΔE/T)

গ) শুধু উপরের মুভে ঘ) সমান সম্ভাবনায়

১০. UCS অপটিমাল যখন খরচ:

ক) ε > 0 হলে খ) ঠিক 1 হলে

গ) ঋণাত্মক হলে ঘ) শূন্য হলে

উত্তরমালা → ১-ঘ · ২-ক · ৩-খ · ৪-গ · ৫-খ · ৬-ঘ · ৭-ক · ৮-গ · ৯-ঘ · ১০-ক

লিখিত প্রশ্ন: নিচের ডিরেক্টেড গ্রাফে এজ খরচ S→A = 2, S→B = 1, A→C = 2, B→C = 4, B→D = 5, C→G = 3, D→G = 2 এবং h(S) = 5, h(A) = 4, h(B) = 4, h(C) = 3, h(D) = 2, h(G) = 0 নিয়ে A* সার্চ চালাও। বিস্তারের ক্রম, পাওয়া পথ ও তার খরচ দাও, আর হিউরিস্টিকটি অ্যাডমিসিবল কিনা তা যাচাই করো। **১০ নম্বর**

১. **f = g + h** ব্যবহার করো, ফ্রন্টিয়ার f-এর ওপর সাজানো প্রায়োরিটি কিউ। শুরু: open = {S (g = 0, f = 5)}। 1

২. **S** বিস্তার: A (g = 2, f = 6), B (g = 1, f = 5); open = {B 5, A 6}। 1

৩. **B** বিস্তার: C (g = 5, f = 8), D (g = 6, f = 8); open = {A 6, C 8, D 8}। 1

৪. **A** বিস্তার (f = 6): C-তে g = 4 < 5 পাওয়া যায়, তাই C **হালনাগাদ** হয়ে g = 4, f = 7; open = {C 7, D 8}। 1

৫. **C** বিস্তার: G (g 7, f 7); open = {G 7, D 8}; **G** বিস্তার — লক্ষ্য পূর্ণ, ক্রম S, B, A, C, G। 2

৬. পথ **S-A-C-G**, খরচ 2 + 2 + 3 = **7**; S-B-C-G = 8 ও S-B-D-G = 8 বেশি। 1

৭. প্রকৃত বাকি খরচ: h*(G) = 0, h*(C) = 3, h*(D) = 2, h*(A) = 5, h*(B) = 7, h*(S) = 7 — প্রতিটি h(n) ≤ h*(n), তাই h **অ্যাডমিসিবল** এবং সামঞ্জস্যপূর্ণও, ফলে A*-এর উত্তর নিশ্চিত **অপটিমাল**। 3

পরীক্ষার আগে যাচাই ✓

- ✓ ফর্মুলেশনের পাঁচটি অংশ লিখতে পারি
- ✓ BFS/DFS-এর সম্পূর্ণ ও অপটিমাল কলাম বলতে পারি
- ✓ A* ও UCS চালিয়ে অ্যাডমিসিবল যাচাই করতে পারি
- ✓ α-β প্রুনিং ও MRV + ফরওয়ার্ড চেকিং করতে পারি